

Autonomous Reckless Driving Detection Using Deep Learning on Embedded GPUs

Taewook Heo^{*,1}, Woojin Nam^{*,2}, Jeongyeup Paek³ *Senior Member, IEEE*, and JeongGil Ko⁴ *Senior Member, IEEE*

^{*}Equally contributing first authors

¹Electronics and Telecommunications Research Institute, ²Department of Computer Engineering, Ajou University

³School of Computer Science and Engineering, Chung-Ang University, ⁴School of Integrated Technology, Yonsei University

Abstract—Reckless driving is dangerous, and must be monitored, detected, and law-enforced to assure road safety. For this purpose, this work presents an embedded system for monitoring and detecting reckless driving activities on the road autonomously in real-time. Using an embedded GPU (eGPU) platform, a camera, and a combination of light-weight deep learning models, we design a system that can identify abnormal vehicle motions on the road. Our system analyzes discrete per-frame images from vehicle detection algorithms, and creates a continuous trace of a vehicle's motion trajectory. While doing so, a virtual grid is generated on the road to obtain positions of vehicles with less overhead and accurately track a vehicle's movement even with low frame rate (5 fps) videos. Vehicle's motion trajectory is then compared against the surrounding to identify abnormal behavior through driving activity classification, which can be provided to law enforcement personnel for final validation. The key challenge is the fundamental resource constraints of embedded platforms, and we design algorithms to overcome their limitations. Evaluation results show that our scheme can well-extract the horizontal and vertical movements of a vehicle (100% recall and 67% precision) and show the potential for truly autonomous reckless driving activity detection systems.

Index Terms—Reckless driving detection; embedded system; deep learning; vehicle motion tracking

I. INTRODUCTION

Vehicle speed detection cameras on the roads today use laser-based sensors or ground-implemented sensors to accurately estimate the speed of a moving vehicle. Once the speed is captured, a picture is taken for vehicles that travel over the limit. The task of these cameras are rather simple, but they take an important role in maintaining road safety. However, installation of ground-implemented sensors can be burdensome, laser-based sensors have high costs, and more importantly, they can capture only the *speed* of a vehicle, only for those that travel within a lane. Nevertheless, speeding is not the only threat we have on the roads of today. A more prominent problem is *reckless driving*, which is the cause of many on-road accidents today. Reckless driving is defined as the overall act of disregarding road rules, and also includes various abnormal driving activities such as frequently and violently changing lanes as well as speeding. However, the camera-based road monitoring systems we have today is incapable

This work was supported by the NRF Grant funded by MSIP (Project No. 2015R1A5A1037668) and KETEP and MOTIE of the Republic of Korea (No. 20182010106460).

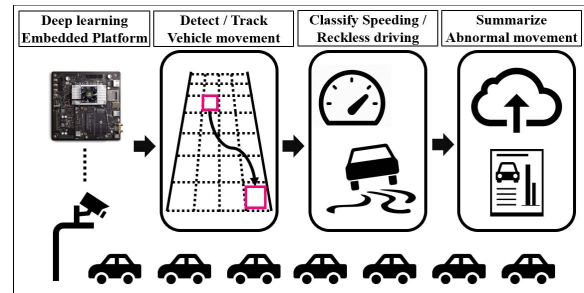


Fig. 1. System architecture

of detecting abnormal vehicular motions comprehensively. Therefore, there is an immense social and economical need for a system that can autonomously detect reckless driving, which not only needs to detect the speed of vehicles but also their movements as well.

In this work, we present an embedded system for autonomously detecting reckless driving activities on the road in real-time. We argue that detecting reckless driving can be done on a stand-alone, low-cost camera-based embedded system with the technology that we have today. By combining camera modules with embedded graphics processing units (eGPUs), we can detect objects (i.e., vehicles) on the road, continuously track their movements, and classify their movements without the need for other sensors nor a powerful backend system, nor high bandwidth video transmissions.

Specifically, Fig. 1 depicts the architecture of our system. A camera-attached eGPU platform deployed on the road performs all necessary computation to classify and identify the motion trajectories of vehicles. Once reckless driving is detected, the device sends alerts to the control center where a law-enforcement professional can make the final decision on vehicle activity. Transmitting all captured videos to a central server requires a significant amount of bandwidth. For example, a 480p quality video typically has a bitrate of 3 Mbps. With 500 cameras, a network bandwidth of 1.5 Gbps is required on the central server. In addition, a huge amount of computation power is spent in processing and analyzing concurrent video streams just to find nothing most of the time. Thus, this work argues that tracking, classification, and

detection should occur on the embedded edge device, and videos of vehicles are sent when and only when abnormal activities are detected. Such a single-unit autonomous system can be deployed just like the speed-limit enforcement cameras we have today, and possibly be incorporated into the road-side units (RSUs) for V2X communication in the future [1].

We also propose several methods to improve the identification of vehicles and estimate vehicular activity on embedded devices. We noticed that the resource constraints (especially the processing power) of eGPUs limit the frame rate at which it can process images, and show that we can design a system that suits our target purpose despite this reduced frame rate. Specifically, the latency of deep learning models for vehicle object detection on an embedded platform (e.g. NVIDIA Jetson TX2) is $\sim 5\times$ longer compared to that on a popular high-end GPU when processing the same video. This limits the rate at which objects can be detected, and thus imposes additional challenge for vehicle tracking. We resolve this issue by tracking adjacent object boxes on subsequent frames, and also by adopting the SURF algorithm [2] to improve tracking accuracy.

Once identifying moving vehicles, the obtained position data from the tracking can be used to analyze the movement of each vehicle. However, since the bounding boxes generated as a result of vehicle detection are not perfectly fit to the vehicles, their raw positions are not adequate to be used as is. Thus, in order to analyze the movement of vehicles consistently, we project the road in a vertical field of view and then create a “virtual grid” to record the vehicle’s positions. By using such schemes, we can measure the speed and movement of vehicles using only the images without any additional sensors. Our proposed reckless driving classification system showed 100% recall and 67% precision based on both real-world and emulated vehicular movement data we use. Although the presence of false positive samples resulted in a relatively low precision, our system showed ‘zero false negatives’ in reckless driving detection.

The contributions of this work are three-fold.

- Through empirical measurements, we show that deep learning models on an eGPU can be implemented to have sufficient performance for designing a reckless driving monitoring system.
- We present a vehicle tracking scheme using object detection results of a deep learning model, and propose various methods to overcome the limited computational power of eGPU platforms.
- We evaluate the proposed system using real-world vehicle traces as well as emulated traces collected from a remote controlled car to show that our scheme can detect reckless driving activities with very high recall.

II. RELATED WORK

Traffic monitoring : There are several prior work in the literature that attempt to monitor road traffic using cameras. However, there are no system (to the best of our knowledge)

that implements deep-learning framework for reckless driving detection. Luvizon et.al [3] use computer vision-based algorithms to recognize and track license plates of vehicles to measure the speed of the vehicles. However, their scheme requires processing high-quality (full-HD) video which is unsuitable for embedded platforms. Furthermore, it cannot recognize vehicles that change directions since front image of the vehicle is necessary, and thus it is not possible to analyze the driving patterns. There are other systems [4]–[6] that do not require high quality videos. However, since these systems monitor the movement of vehicles by tracking only within a specific lane, it is not possible to track and analyze the vehicle’s horizontal movements. On the contrary, our system detects reckless driving by monitoring and analyzing the movement of vehicles on the entire road.

Object detection on embedded device : Since most of the current state-of-the-art deep learning frameworks are not intended for operating in embedded systems, an accurate measurement study is needed to determine the performance of those frameworks on resource-constrained platforms. The work in [7], [8] are surveys on performance measurements of embedded platforms, especially NVIDIA Jetson TX1. There are also applications that use deep learning frameworks in embedded platforms [9]–[14], but mostly on smartphones. The common goal of these works is to utilize deep learning framework efficiently in their systems, even on low performance platforms. Our work examines the performance of the embedded platform, NVIDIA Jetson TX2, for running image processing and deep learning models, and discusses system design. Although newer, faster, and more expensive Xavier platform is released at the time of this writing, newer server-class GPUs have been released as well; we believe our findings and methodology still provides meaningful insights to the relative performance difference and challenges of embedded GPU platforms.

Handling low performance : The eGPU platform used in this work is not as capable as high-end GPUs. Therefore, frame rate of the videos needs to be lowered for real-time processing. Our work uses 5 frames per second (fps), and several other works in [15]–[19] have also used the same rate to implement their systems for the same reason. Furthermore, the position data of a detected object obtained by the deep learning framework is not perfect, and thus calibration is required. Zhou et.al [20] uses a filter to calibrate the positions of moving objects, but low frame rate video is not suitable for such filtering. Thus, we use clustering of positions by generating grid on images through a projection transformation of the road image, similar to ideas in [21], [22]. However, none of the aforementioned work detects reckless driving behavior.

III. PRELIMINARY STUDIES

Our goal is to design an automated system for detecting reckless driving activities on the road via a stand-alone camera-based embedded platform. There are many recent work that utilize deep learning algorithms to detect vehicles and

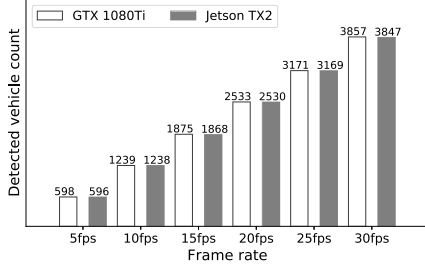


Fig. 2. Comparison of vehicle detection performance between high-end GPU and embedded GPU.

extract their location information on high-end GPUs [23], [24]. However, the key challenge is to design the classification algorithms (for detecting vehicle objects and their movement patterns) to be “light-enough” to properly function on embedded GPU platforms. A vehicle monitoring system requires fast processing to capture and analyze the movement of multiple vehicles on the roads in *real-time*, and thus the performance of the deep learning model is one of the most important factors in implementing our target system. For this reason, we first conducted a preliminary study to investigate the performance of deep learning algorithms operating on eGPUs.

Among various deep learning models optimized for fast object detection [25]–[27], we have selected and compared the processing speeds of SSD (SSD500) [26] and YOLO (YOLOv2 416x416) [27] that exhibit a tradeoff between the object detection accuracy and processing latency. Specifically, we used a 30 fps 480p video and define systems that can process such videos as “real-time capable”. We then compared the object detection rate of SSD and YOLO models on an NVIDIA 1080 Ti GPU. Our results were consistent with previous work: we were able to see a *frame processing rate* (fpr) of 14.7 and 30 on the SSD and YOLO models, respectively. This suggests that YOLO is more light-weight and can achieve higher fpr, which is a critical factor for us given that we plan to implant the model on resource-constraint platforms. For such reasons, we selected the YOLO (v2) model as our deep learning model for vehicle detection.

A. Vehicle object detection

Deep learning frameworks such as YOLO utilize general purpose GPUs to perform their large amount of computation. Embedded systems can also mimic such strong computational capabilities by utilizing eGPU platforms. Among many available, in this work, we utilize the NVIDIA *Jetson TX2* as our eGPU platform. Despite being one of the most high-performance eGPU in the market today, Jetson TX2 has only 7% of CUDA cores compared to server-scale GPUs such as the NVIDIA GTX 1080Ti (Jetson TX2: 256 cores, GTX 1080Ti: 3584 cores). This suggests that the performance of Jetson TX2, although being a GPU as well, will not match that of the GTX 1080Ti. Therefore, it is important to understand the limitations of the eGPU in utilizing the YOLO deep learning framework in designing our system.

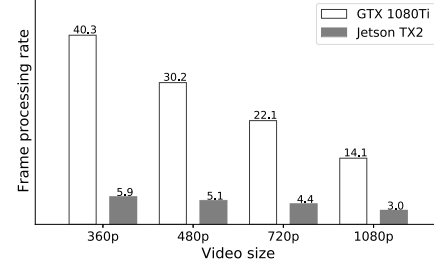


Fig. 3. Detection speed with respect to video size

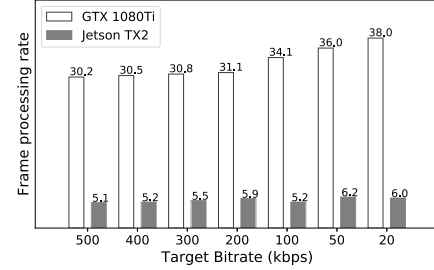


Fig. 4. Detection speed vs. video quality (target bitrate)

Using the performance of the YOLO deep learning model operating on the GTX 1080Ti as the baseline performance, we compare the results of the same model operating on the Jetson TX2 to empirically understand the limitations of eGPUs.

Firstly, given that a highly accurate vehicle object detection on the road is the basis of our reckless driving detection system, we start by confirming if the vehicle detection results for the two platforms match. Fig. 2 presents the vehicle detection instances when the YOLO model operates on each GPU platform while changing the frame rate of the input video which consists of multiple vehicles passing through on a highway for one minute. Note that, with a lower frame rate, less number of vehicles may be detected given reduced number of images that the GPU analyzes for a fixed amount of record time. We can see from the figure that the results are ~99% identical for the two platforms, which suggests that using a more computationally constrained GPUs did not affect the object detection count (accuracy) of vehicles on the road.

B. Video characteristics

Next, we examine the impact of video size or quality on processing speed or detection accuracy for the two GPU platforms. Results from such experiments can help determine the appropriate frame rate for our eGPU-based system.

Fig. 3 plots the processing speed of the deep learning framework with respect to the image size of the video on the GTX 1080 Ti and Jetson TX2 platforms. Here, the target bitrate of the video is fixed to 500 kbps. The results suggest that the processing speed tends to decrease as the size of the image size increases. This is somewhat expected given that a larger size video will take more time to process on a per-frame-basis. As mentioned above, we define 30 fpr as real-time; thus,

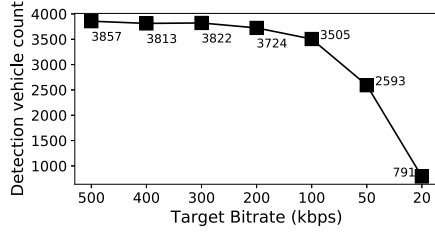


Fig. 5. Detection performance with respect to video quality

for the GTX 1080Ti, we configured the video image size to at least 480p (and meet 30 fpr).

Fig. 4 shows how the processing speed varies with video quality (target bitrate) of the 480p size video. We can see that the quality of the video does not lead to a drastic effect on processing speed unlike the image size. Decreasing the video quality to 20% of the original (500 vs. 100 kbps) only shows a marginal increase in the number of frames that the GPU can process. However, if the video quality is lowered to this level, the performance of vehicle detection, as we show in Fig. 5, where we present the vehicle detection count for different target bitrates on a 480p video, starts to degrade. This suggests us with a minimum target bitrate quality for the videos we use in this work. Finally, we decided to implement the system based on the video with the quality of 500 kbps, which is the original one, given that the gain due to lowering the video's target bitrate is not considered to be significant.

The surprising factor here is that despite having similar vehicle object detection counts (Fig. 3), increasing video size forces the Jetson TX2 eGPU to suffer from a low throughput performance. Unfortunately, the Jetson TX2, compared to the GTX 1080Ti, can only process 14-21% of the frames in a given amount of time. Quantitatively, for the case with 480p video size, the Jetson TX2 can only process 5 fpr. This implies that a full real-time system cannot be implemented at 30 fps, but the same performance needs to be achieved using videos at 5 fps. This can be a problem given that the detection quantity of vehicles can decrease, which naturally leads to coarse-grained vehicular tracking compared to processing video frames at 30 fpr. The following sections will discuss how we design a system to overcome such limitations that the combination of deep learning models and image-based sensing embedded systems introduce.

IV. SYSTEM DESIGN

We now present the design of our system and show details in its implementation. Our autonomous reckless driving detection scheme consists of two core components: (1) a vehicle tracking module, which keeps track of vehicles in a video from the frame that the vehicle first appears until it disappears, and (2) a vehicle motion and speed extraction module, which, using the tracking data, tries to identify specific motion trajectories to extract the speed and motion patterns. We detail the design of these two major components using the following sections.

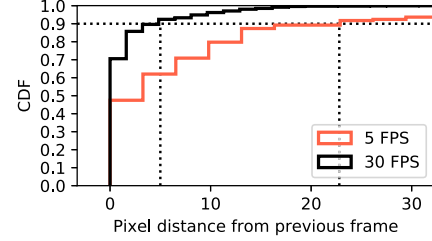


Fig. 6. Cumulative Distribution Function (CDF) of pixel distances between detected bounding boxes in subsequent frames.

A. Vehicle tracking

The YOLO model performs vehicle detection on a per-frame basis. Therefore, we need a scheme to confirm that two independently identified vehicles in subsequent frames are the same vehicle to perform a reliable and continuous movement tracking. Note that the result of a vehicle object detection by YOLO, as with most object recognition algorithms, outputs a bounding box of the target object. In our system, we store the center point of each object box to measure the distance between center points of subsequent frames. Using this information, we determine if the distance (between two frames) is within a preset threshold to identify if these boxes cover the same vehicle. However, since the moving distance in subsequent frames is larger in a lower frame rate videos (due to increased interval between each subsequent frames), it is necessary to apply a different detection range threshold adaptively with respect to the current video frame rate. Furthermore, given that the detection accuracy of YOLO is imperfect and may miss objects in a frame or even miss the frame itself due to delayed processing, a naive scheme that simply uses a threshold between two center point pairs to track the nearest box may result in tracking wrong/different vehicles. Therefore, rather than simply tracking the nearest boxes, we need to configure a maximum allowable range of detection and consider only the boxes (center point) within that range to be considered as the same vehicle as we track vehicular movements.

To determine the range threshold parameter for each frame rate, we measured the pixel distances between the nearest boxes in subsequent frames. Fig. 6 presents a CDF of the pixel distances for objects detected in subsequent frames when using 5 and 30 fps videos. The video used here consists of vehicles moving from top to bottom of the frame, and the data was collected from total of 13 different road monitoring videos to prevent overfitting in the model training phase. Of the pixel distance distributions collected from each video, we identified that 90% of the data were detected as a subset of a single vehicle's movement trajectory. The other 10% may or may not be other vehicle(s). Furthermore, the figure shows that the distance distribution differs by as much as 18 pixels when comparing 90% probability for the two frame rates (i.e., 23 for 5 fps and 5 for 30 fps). This result itself seems promising that, having 5 fps will not hurt the system

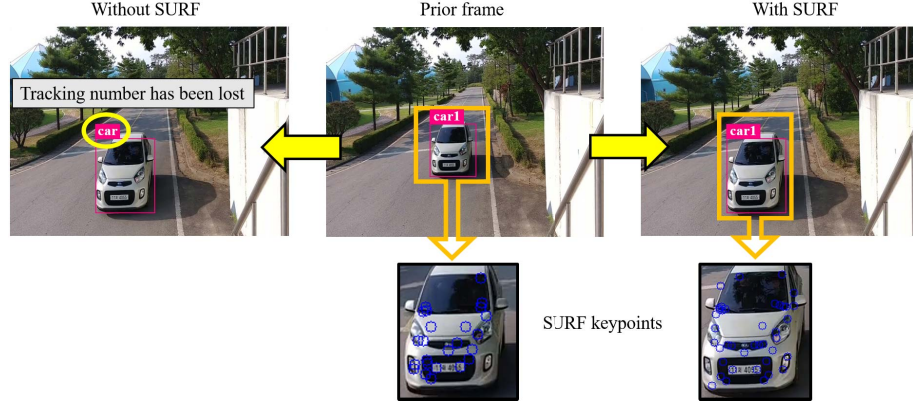


Fig. 7. An example of how SURF is applied in our system. The center image is the frame $n - 1$ and the images on both sides are the subsequent frames (frame n). When not using SURF (left), continuous vehicle tracking fails. When using SURF (right), continuous tracking takes place. Two bottom images show the SURF keypoints detected in each frame.

performance significantly when the vehicle tracking range threshold is properly configured. However, these results are for cases when vehicles are all properly detected at each frame. As mentioned, there can also be cases where frames are missed or objects are missed from the image. Under such circumstances, with lower frame rates, the pixel distance threshold between adjacent frames may need to be increased considerably.

Based on such observations, we use a frame buffering scheme where we keep track a window of five (for 30 fps systems and two for 5 fps systems) most recent “object detected” frames. Specifically, we store the center point information for the objects detected in the current window and use this information to infer a vehicle’s trajectory.

Finally, to avoid the center points from exceeding the vehicle tracking range threshold, we apply the SURF algorithm [2] to compensate for potential errors. Specifically, we store not only the center point information of the buffered frames as mentioned above, but also the *keypoint* information of the vehicles’ image obtained from the SURF algorithm. These keypoints in SURF are the edge information that characterizes the shape of the object within the image. For example, the images at the bottom of Fig. 7 show SURF keypoints of the vehicle obtained from our test videos. If there is no position information for tracking a specific vehicle identified from the buffered frames, the “keypoint similarity” matching between the buffered keypoints and the target keypoints is performed. At this point, we track the vehicle with the highest matching score in terms of keypoint similarity. Fig. 7 shows an example of a tracked vehicle with and without the SURF algorithm. The figure shows that, without SURF, our system would miss the fact that the two subsequent frames are showing the same object (the car tracking number - *car 1* - is lost). However, when using SURF, we compare the SURF keypoints to continuously track *car 1*.

Overall, our scheme combining the empirically measured vehicle tracking range threshold for 5 fps and SURF achieves 99.8% vehicle tracking accuracy for our preliminary dataset

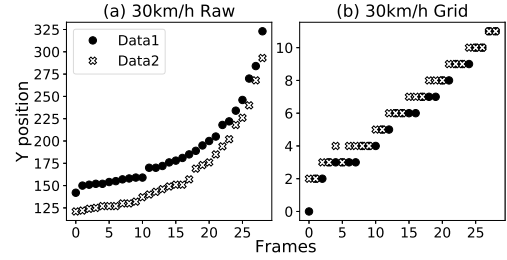


Fig. 8. Per frame y-axis positions for two driving data samples. The left uses raw data and the right side uses a projected grid.

(89 successful from 90 vehicle traces collected from real road environments with many vehicles operating simultaneously).

B. Movement Analysis

While the most intuitive way to analyze vehicle movement may be to use the center points from the vehicle tracking algorithm directly, this does not provide accurate results due to object center point fluctuations, since the object bounding boxes may not fit the vehicle perfectly due to the fast but coarse-grained computing characteristics of YOLO. This fluctuation may seem minute from a wide-scaled vision, but when observing details in pixel-scale, small fluctuations can lead to large amounts of estimation error. As a result, analyzing only the center-point information cannot guarantee reliable results even when the vehicle (in reality) shows a very uniform movement. Furthermore, note that the vehicle’s object size increases as the object moves closer towards the camera, which is typically installed to be tilted and facing down the road from a fixed location. Since the same number of pixel motion when the vehicle is nearby and far away represent different distances, such factors should be taken in consideration to improve the system accuracy.

Fig. 8 (a) plots the y-axis positions for the two videos (raw vs. projected) each traveling at 30 km/h captured using 5 fps. We notice that as frames proceed, the y-axis location of the

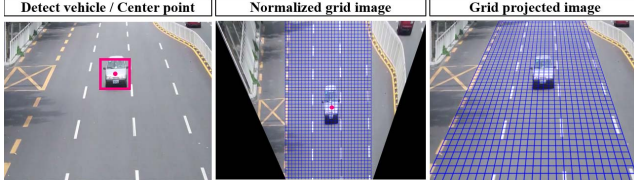


Fig. 9. Grid presentation of images captured from road monitoring cameras.

vehicle increases super-linearly. Also, some points are missing due to the imperfect nature of the YOLO object detection algorithm. We tried approaching this issue in two different ways. First, we applied a sliding window to minimize the effect of missing frames. While this seemed like a plausible approach and improved the performance for 30 fps videos, we noted that it had minimal effect in improving the 5 fps video's performance due to the fact that the box distance between subsequent frames are farther away in the 5 fps videos even for identical vehicle velocity.

As the second approach, we clustered the location information of vehicular objects extracted from the 5 fps video and use a "binning-based" pre-processing approach. Specifically, given that road monitoring cameras are not vertically positioned in the sky but rather tilted towards the road as shown in Fig. 9, we re-project the road to make a normalized rectangular grid of the current road being monitored using the following equation. By doing so, we sacrifice location accuracy's fidelity, but such errors can be improved by controlling the grid granularity. Compared to Fig. 8 (a) where the raw points are plotted, in Fig. 8 (b) we can see that by applying the 2D grid projection the y-axis positions of the two vehicles start to merge together. This merge means that the speed estimation (which is the same for both vehicles) can be made more accurate.

The grids used in our scheme is projected based on the following equation.

$$\begin{bmatrix} t_i x'_i \\ t_i y'_i \\ t_i \end{bmatrix} = \text{map_matrix} \cdot \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix} \quad (1)$$

Here, t_i is a constant and x_i, y_i are the original x and y coordinates with x'_i, y'_i being the corresponding coordinates on the normalized grid (with a different view point). The map_matrix is a matrix that changes the reference point of view when converting a set of points for different view points based on the homogeneous matrix system [28]. We can calculate map_matrix through four coordinates that are to be projected from the original image, and also using four coordinates of where the original coordinates will be positioned after the projection. We use the area of the road and select four coordinates based on the traffic lanes. These four corner coordinates are reflected to the projected image to extract the full size of the original image. Once map_matrix for a specific image is obtained, the coordinates of the normalized image can be calculated by putting the coordinates of the original image into the aforementioned formula.

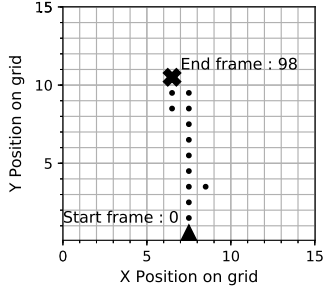
Such an approach allows us to design an algorithm that is light-weight enough to run on embedded devices. Note once more that the grid cell size will limit the accuracy of both speed and lane change actions. Since reckless driving is usually shown in the form of frequent change of lanes *and* speeding, maintaining accuracy is important. We found from our empirical studies that for a vehicle captured with a size of 30x30 pixels can be well represented with a grid size of 40x40 for both 30 fps and 5 fps systems, and this value can be easily adjusted on a per-deployment basis. While we will discuss more in the evaluations, the resulting projection of a vehicle's movements are shown as Figs. 10, 11, and 12 in our system. We use this information to extract a coordinates as presented in Fig. 8 (b) and perform linear regression on this data to extract the vehicle speed. We present an example of such regression result in Fig. 13 for two video samples with a vehicle operating at 30 km/h. While the regression results are not identical, we see that this contains enough similarity to make decisions on the "speed range" of the captured vehicles.

Furthermore, using the grid information, we not only use the y-axis data for estimating speed, but we can exploit the x-axis data to make estimations on lane change events, which are both important for detecting reckless driving behaviors. Since the criteria for reckless driving vary from country to country, state to state, and even between the judgments of the law-enforcement officers, we focus on detecting 'unusual' driving patterns rather than proposing a model for determining the recklessness of vehicles.

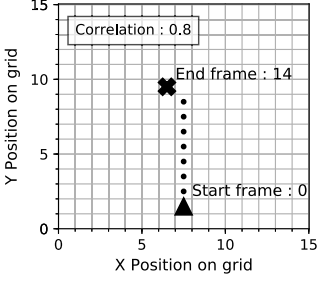
Nevertheless, reckless driving detection should be able to differentiate between "aggressive lane changes" and "normal lane changes". Therefore, we add in the concept of lane change frequency and latency to capture how often and how fast the vehicle changes lanes. In doing so, we gather many normal driving activity samples and define the confidence region for lane change (horizontal movement) variance. For input data that are classified as outliers (e.g., higher than the 99.9% confidence interval), we set these vehicles as potential reckless driving samples, and compare the speed range of the vehicle with the speed limit of the road.

V. EVALUATION

We evaluate our proposed system using two types of videos. First, we use a set of videos for a real vehicle on the road. This video mostly drives in a straight line, with varying speeds while obeying the road regulations. Second, we use a set of videos captured using a remote controlled vehicle on a sample road-like condition. Given that reckless driving sample videos cannot be self-captured without disobeying the road regulations, we use a remote controlled vehicle to emulate such situations. For both cases, the videos were captured at 30 fps and we uniformly down-sample the videos to create their respective 5 fps versions. While there are openly available roadside video samples, we use self-generated samples to accurately know the ground-truth of all effects that occur on the video. We use the NVIDIA Jetson TX2 eGPU as our embedded platform for all experiments unless specified.

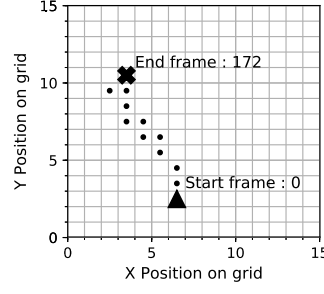


(a) 30fps

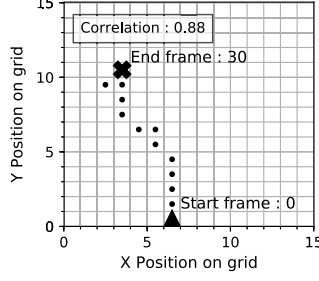


(b) 5fps

Fig. 10. Driving straight example

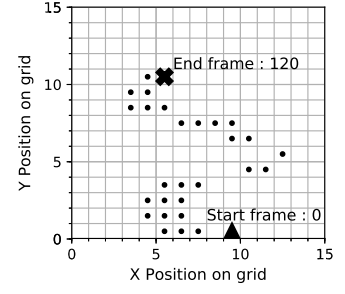


(a) 30fps

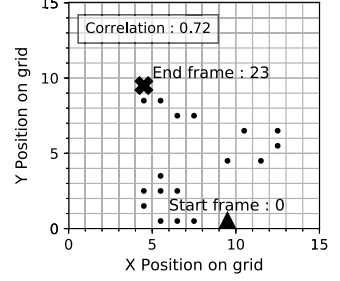


(b) 5fps

Fig. 11. Smooth lane change example



(a) 30fps



(b) 5fps

Fig. 12. Reckless driving example

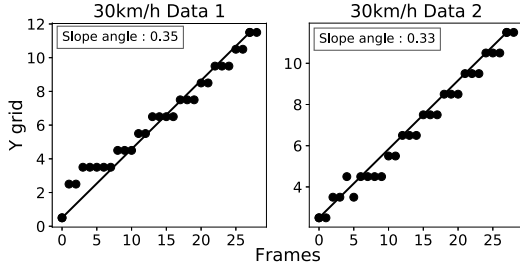


Fig. 13. Slope angle for extracting speeds from two sample video data.

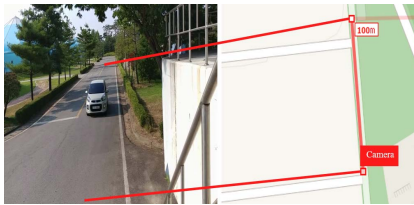


Fig. 14. Sample video for real-world data.

Fig. 14 presents a scene of our real-world traffic collection. The straight road in the figure is approximately 100 meters long and we set the vehicle to cruise at 30, 40, 50, 60 and 70 km/h. The remote controlled car testing environment is presented in Fig. 15. In this testing phase, we made sure that the remote controlled car movements well-mimicked real vehicles motions. Fortunately, given that the remote controlled vehicle's appearance is very similar to an actual vehicle, the object detection accuracy from the YOLO model did not differ for the real vehicle and the remote control vehicle.

Once the vehicle is projected on the grid, its speed can be

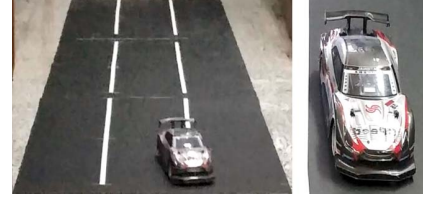


Fig. 15. Sample video for remote controlled vehicle data.

computed using the time duration and the y-axis grid trajectories. Lane change can also be detected using the horizontal (x-axis) movement of the vehicle. These two features together allows our system to easily detect reckless driving behaviors. Figs. 10, 11, 12 present examples of how the speed and lane change information can be extracted from the grid created in our system for three different data samples making different types of movements: when the vehicle drives straight, when changing lanes, and a sample of reckless driving, respectively.

While the target frame rate of our embedded system is 5 fps, we also present the results with 30 fps for performance comparison purposes. The results in 30 fps are used as the ground truth of the 5 fps system. If the path data on the grid to be used for analysis is not sufficiently accumulated by using low frame rates, it would be difficult to design a 5 fps video-based system. Therefore, we see the need to compare the tracking results in 5 fps settings with that of 30 fps videos.

Note that in these three figures that the start and end points slightly differ when comparing the two frame rates. This difference is caused by the difference in the frame rate itself given that the starting position will depend on the granularity of the video. For example, in case of driving straight (Fig. 10),

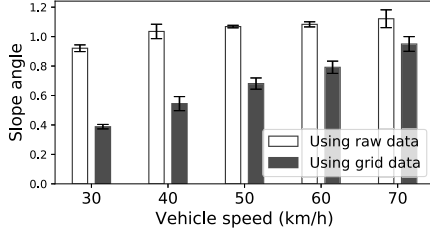


Fig. 16. Slope angles (regression values) for each speed range of a vehicle.

Drive straight	Lane change	Reckless driving	Total
0.81	0.80	0.72	0.78

TABLE I
CORRELATION COEFFICIENT BETWEEN 30 AND 5 FPS VIDEOS.

the position of the start frame is located at (7,0) in the 30 fps sample, but in (7,1) for the 5 fps sample. Similarly, the end points are (6, 11) for 30 fps but (6, 10) when using the 5 fps video. Nevertheless, the effect of such small differences in detection points are diminished by the time differences that the two videos possess.

In any case, the correlation between the two trajectories for the three driving conditions are higher than 0.75. Nevertheless, note that the correlation for the reckless driving case (Fig. 12) is the lowest among the three. This is an effect of the 30 fps video offering “too fine-grain” measurements. As a vehicle makes turns, the heading direction changes along with the size of the bounding box. This generates overlapping points at same coordinates, causing differences with the 5 fps video.

Table I summarizes correlations between the results for 30 fps videos and 5 fps videos across all our test data. These results, again, show that 5 fps is sufficient in reflecting the details that 30 fps videos possess for analyzing vehicle trajectory characteristics.

As discussed in Section IV, the speed estimation algorithm uses the differences of the y-axis coordinates on the projected 2D grid. Using these coordinates, we extract the slope angle when applying linear regression on the grid samples. We evaluated how the grid coordinates can help estimate the speed of a vehicle using a total of 15 videos consisting of five samples for each speed between 30-70 km/h. Ideally, as we test for increasing speeds, the slope of the linear regression will become more steep. Since our goal is to estimate the speed range to capture reckless driving activities, we used speed ranges in steps of 10 km/h.

Fig. 16 presents the average of the regression values for each speed range of a vehicle. Notice that when using raw coordinates (center points) extracted from the 5 fps video, the linear regression slope angle does not show a linear increase despite the speed linearly increasing. Fortunately, when applying the grid-based scheme to mitigate object detection errors, we can see a linearly increasing slope as the vehicle speeds increase linearly. This infers that our grid-based speed estimation scheme can be an effective way of predicting the vehicular speed. Unfortunately, due to the granularity in controlling the vehicle speed in normal road conditions, we

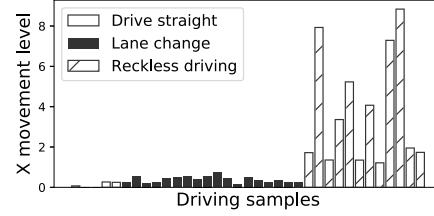


Fig. 17. X-axis motion levels for all test data samples.

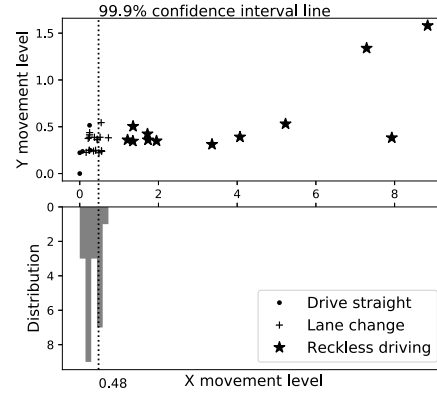


Fig. 18. The distribution of the movement level for each driving pattern. Vertical line represents the cut off for reckless driving detection.

were not able to test for the speed in between the 10 km/h steps. Nevertheless, the mean linear regression slope angles and their standard deviations suggest that our scheme can potentially achieve a finer granularity in speed estimation.

We note that the linear regression slope angles for detecting speed will vary with camera installation height and angle, and this can be done by adjusting threshold parameters in the installation phase and is a one-time effort.

While the y-axis coordinates hint us in detecting the vehicle speed, to detect reckless driving activities, the x-axis data must be exploited to detect frequent and sudden lane changes. Specifically, we focus on the amount of variation occurring on the x-axis coordinates to for such predictions. In our system, we calculate the x-axis variation as a string of changes by measuring the amount of grid movement in the x-axis compared to the previous frame. In other words, if the vehicle is detected to have moved to the right by one column, we add 1 to the variation string, and if it moved to the left by two, we add -2. We then compute the variance of this string of movements to quantify the level of vehicular horizontal motions. This movement level is presented in Fig. 17.

In the case of straight-driving or a “smooth” lane change, vehicles show low motion levels and the measurements are distributed within a certain range (mean 0.33, stdev 0.15). On the other hand, for reckless driving, a wider range of motions occur. Again, as mentioned above, we classify reckless driving motions as samples that show more than $\pm 3.3\sigma$ (99.9% confidence interval) of the normal driving vehicles. Fig. 18 presents how the samples can be clustered in the 2D space of

Grid (px)	TP	FP	FN	TN	Accuracy	Precision	Recall
20x20	12	8	0	15	77%	60%	100%
40x40	12	6	0	17	83%	67%	100%
60x60	12	3	0	20	91%	80%	100%

TABLE II
CONFUSION MATRIX OF ABNORMAL DRIVING CLASSIFICATION WITH
RESPECT TO THE GRID SIZE.

x- and y-axes motion levels and we also present the 99.9% confidence interval for normal driving data. Quantitatively, as Table II shows, the recall (detecting reckless vehicles as reckless driving samples) is 100% regardless of grid sizes, while the lower precision shows that there are some false positives (normal vehicles as reckless driving samples).

We see this result better than missing the detection of reckless driving behavior given our application characteristics where, at the end of the day, the final decision on reckless driving should be made by a professional law enforcement personnel. Our system assists such decisions by reducing the set size for the professionals to evaluate. As future work, we plan to extend and improve our studies with more samples to reduce the recall as a way to minimize the labor cost of professionals that declare reckless driving behaviors.

VI. CONCLUSION

This work presented a camera-based embedded sensing system for autonomously detecting reckless driving activities on the road in real-time. Specifically, we exploited the capabilities of the YOLO deep learning model and designed a system that can operate on resource-limited embedded platforms by combining a grid projection based approach with the SURF algorithm. Using an implementation on the NVIDIA Jetson TX2 embedded GPU platform, we show that even simple classification methods can achieve a recall rate of 100% using video clips collected from real roads and emulated settings. As part of our future work we plan to perform a pilot deployment of the system on real roads to accurately evaluate the system's performance under practical settings.

REFERENCES

- [1] J. Heo, B. Kang, J. M. Yang, J. Paek, and S. Bahk, "Performance-Cost Tradeoff of Using Mobile Roadside Units for V2X Communication," *IEEE Transactions on Vehicular Technology*, vol. 68, no. 9, pp. 9049–9059, 2019.
- [2] H. Bay, A. Ess, T. Tuytelaars, and L. V. Gool, "Speeded-Up Robust Features (SURF)," *Computer Vision and Image Understanding*, vol. 110, no. 3, pp. 346 – 359, 2008.
- [3] D. C. Luvizon, B. T. Nassu, and R. Minetto, "A Video-Based System for Vehicle Speed Measurement in Urban Roadways," *IEEE Transactions on Intelligent Transportation Systems*, vol. 18, no. 6, pp. 1393–1404, June 2017.
- [4] T. Celik and H. Kusetoğlu, "Solar-Powered Automated Road Surveillance System for Speed Violation Detection," *IEEE Transactions on Industrial Electronics*, vol. 57, no. 9, pp. 3216–3227, Sept 2010.
- [5] T. T. Nguyen, X. D. Pham, J. H. Song, S. Jin, D. Kim, and J. W. Jeon, "Compensating Background for Noise due to Camera Vibration in Uncalibrated-Camera-Based Vehicle Speed Measurement System," *IEEE Transactions on Vehicular Technology*, vol. 60, no. 1, pp. 30–43, Jan 2011.
- [6] C. Arth, H. Bischof, and C. Leistner, "TRICam - An Embedded Platform for Remote Traffic Surveillance," in *Conference on Computer Vision and Pattern Recognition Workshop (CVPRW'06)*, June 2006, pp. 125–125.
- [7] C. Rodrigues, G. Riley, and M. Luján, "SyNERGY: An energy measurement and prediction framework for Convolutional Neural Networks on Jetson TX1," in *Int'l Conf on Parallel and Distributed Processing Techniques and Applications*, 10 2018.
- [8] A. Canziani, A. Paszke, and E. Culurciello, "An Analysis of Deep Neural Network Models for Practical Applications," *CoRR*, 2016. [Online]. Available: <http://arxiv.org/abs/1605.07678>
- [9] T. Y.-H. Chen, L. Ravindranath, S. Deng, P. Bahl, and H. Balakrishnan, "Glimpse: Continuous, Real-Time Object Recognition on Mobile Devices," in *ACM Conference on Embedded Networked Sensor Systems (SenSys'15)*, 2015, pp. 155–168.
- [10] P. Wang, Q. Zhang, L. Li, F. Ru, D. Li, and Y. Jin, "Deep learning-based gesture recognition for control of mobile body-weight support platform," in *IEEE Conference on Industrial Electronics and Applications (ICIEA)*, May 2018, pp. 1803–1808.
- [11] R. Tanno, K. Okamoto, and K. Yanai, "DeepFoodCam: A DCNN-based Real-time Mobile Food Recognition System," in *Proceedings of the 2nd International Workshop on Multimedia Assisted Dietary Management (MADiMa'16)*, 2016.
- [12] K. Ota, M. S. Dao, V. Mezaris, and F. G. B. D. Natale, "Deep Learning for Mobile Multimedia: A Survey," *ACM Trans. Multimedia Comput. Commun. Appl.*, vol. 13, no. 3s, pp. 34:1–34:22, June 2017.
- [13] W. Liu, H. Ma, H. Qi, D. Zhao, and Z. Chen, "Deep learning hashing for mobile visual search," *EURASIP Journal on Image and Video Processing*, vol. 2017, no. 1, Feb 2017.
- [14] D. Tomè, F. Monti, L. Baroffio, L. Bondi, M. Tagliasacchi, and S. Tubaro, "Deep Convolutional Neural Networks for pedestrian detection," *Signal Processing: Image Communication*, vol. 47, pp. 482 – 489, 2016.
- [15] M. Chuang, J. Hwang, K. Williams, and R. Towler, "Tracking Live Fish From Low-Contrast and Low-Frame-Rate Stereo Videos," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 25, no. 1, pp. 167–179, Jan 2015.
- [16] K. Fang, T.-L. Wu, D. Yang, S. Savarese, and J. J. Lim, "Demo2Vec: Reasoning Object Affordances From Online Videos," in *IEEE CVPR*, June 2018.
- [17] W. Min, Y. Zhang, J. Li, and S. Xu, "Recognition of pedestrian activity based on dropped-object detection," *Signal Processing*, vol. 144, pp. 238 – 252, 2018.
- [18] S. Xu, A. Savvaris, S. He, H. Shin, and A. Tsourdos, "Real-time Implementation of YOLO+JPDA for Small Scale UAV Multiple Object Tracking," in *International Conference on Unmanned Aircraft Systems (ICUAS)*, June 2018, pp. 1336–1341.
- [19] E. Toropov, L. Gui, S. Zhang, S. Kottur, and J. M. F. Moura, "Traffic flow from a low frame rate city camera," in *IEEE International Conference on Image Processing (ICIP)*, Sept 2015, pp. 3802–3806.
- [20] Z. Zhou, D. Wu, and Z. Zhu, "Object tracking based on Kalman particle filter with LSSVR," *Optik*, vol. 127, no. 2, pp. 613 – 619, 2016.
- [21] Y. Wu and Z. Chen, "A detection method of road traffic sign based on inverse perspective transform," in *IEEE International Conference of Online Analysis and Computing Science*, May 2016, pp. 293–296.
- [22] H. Palaio, C. Maduro, K. Batista, and J. Batista, "Ground plane velocity estimation embedding rectification on a particle filter multi-target tracking," in *IEEE International Conference on Robotics and Automation*, May 2009, pp. 825–830.
- [23] I. Baek, A. Davies, G. Yan, and R. R. Rajkumar, "Real-time Detection, Tracking, and Classification of Moving and Stationary Objects using Multiple Fisheye Images," in *IEEE Intelligent Vehicles Symposium (IV)*, June 2018, pp. 447–452.
- [24] R. Ke, Z. Li, J. Tang, Z. Pan, and Y. Wang, "Real-Time Traffic Flow Parameter Estimation From UAV Video Based on Ensemble Classifier and Optical Flow," *IEEE Transactions on Intelligent Transportation Systems*, pp. 1–11, 2018.
- [25] S. Ren, K. He, R. B. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," *CoRR*, 2015. [Online]. Available: <http://arxiv.org/abs/1506.01497>
- [26] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. E. Reed, C. Fu, and A. C. Berg, "SSD: single shot multibox detector," *CoRR*, 2015. [Online]. Available: <http://arxiv.org/abs/1512.02325>
- [27] J. Redmon and A. Farhadi, "YOLO9000: Better, Faster, Stronger," *arXiv preprint arXiv:1612.08242*, 2016.
- [28] H. Li, D. Hestenes, and A. Rockwood, *Generalized Homogeneous Coordinates for Computational Geometry*. Springer Berlin Heidelberg, 2001, pp. 27–59.